

Project Documentation: BirdCLEF 2024

Audio Classification

Project Title: BirdCLEF 2024 Species Classification

Framework: Keras 3 with JAX Backend

Model Architecture: EfficientNetV2-B2

1. Project Overview

This notebook implements a deep learning pipeline designed for the **BirdCLEF 2024** competition. The objective is to identify bird species from audio recordings collected in India. The solution transforms raw audio signals into Mel-spectrograms (image representations) and leverages a Convolutional Neural Network (CNN) to perform multi-class classification across 182 distinct species.

2. Environment and Configuration

2.1 Backend and Libraries

The code is optimized for performance using the **JAX** backend for Keras 3, while utilizing TensorFlow for data I/O.

- **Key Dependencies:** keras_cv, tensorflow, tensorflow_io, librosa, numpy, pandas.
- **Runtime:** Configured for GPU acceleration (T4 recommended).

2.2 Global Configuration (CFG)

A centralized configuration class manages hyperparameters to ensure consistency across the pipeline:

- **Audio Parameters:** 15s duration, 32,000 Hz sample rate.
- **Image Processing:** Spectrogram size of [128, 384].
- **STFT Parameters:** nfft = 2028, window = 2048, fmin = 20Hz, fmax = 16,000Hz.
- **Training:** Batch size of 16, 5 epochs, and EfficientNetV2-B2 backbone initialized with ImageNet weights.

3. Data Engineering

3.1 Dataset Setup

The system is designed to load data directly from Google Drive. It expects a specific directory structure:

```
birdclef-2024/  
├── train_audio/ (Folders named by species ID)  
└── train_metadata.csv
```

The script validates the existence of 24,459 samples and maps the 182 folder names to integer labels for model training.

3.2 Feature Extraction (Spectrograms)

To process audio with a computer vision model, the code converts 1D waveforms into 2D Mel-spectrograms using librosa:

1. **Loading:** Audio is loaded and trimmed/padded to a fixed length (15 seconds).
2. **Mel-Spectrogram Generation:** Uses Log-Mel scaling to represent frequencies similarly to human hearing.
3. **Normalization:** Min-Max scaling is applied to ensure pixel values are between 0 and 1, facilitating faster model convergence.

3.3 Visualization Utilities

A dedicated `display_audio` function is provided for exploratory data analysis (EDA). It renders:

- An interactive audio player.
- The raw waveform (amplitude over time).
- The resulting Mel-spectrogram (frequency over time) using the coolwarm colormap.

4. Model Architecture

The pipeline utilizes **EfficientNetV2-B2**, a highly efficient CNN architecture provided by KerasCV.

Component	Description
Backbone	efficientnetv2_b2
Weights	Pre-trained on ImageNet
Input Shape	(128, 384, 3)
Output Layer	Softmax dense layer with 182 units (Species count)

The use of a pre-trained backbone allows the model to leverage "transfer learning," utilizing general image patterns (edges, textures) learned from ImageNet to better understand the patterns within spectrograms.

5. Training Pipeline

5.1 Augmentation

The configuration includes an `augment=True` flag. While specific augmentation layers (like SpecAugment or Mixup) are initialized through KerasCV, they are typically applied to the training dataset to prevent overfitting by artificially increasing the diversity of the audio samples.

5.2 Reproducibility

To ensure experiments can be replicated, the notebook sets a global random seed (42) for both Python and TensorFlow's random number generators.

6. Usage Instructions

1. **Mount Drive:** Ensure the dataset is uploaded to your Google Drive in the path `/Deep_Learning_NHF/birdclef-2024/`.
2. **Initialize Backend:** Ensure `os.environ["KERAS_BACKEND"] = "jax"` is set before importing Keras.
3. **Run Pipeline:**
 - Execute the **Configuration** cell to set hyperparameters.
 - Execute the **Utility** cells to define audio loaders.
 - Execute the **Training** cells (if present) or use the defined CFG to build a `model.fit()` loop.

7. Troubleshooting

- **Path Errors:** Verify the `DRIVE_DATASET_PATH` variable matches your Drive folder structure.
- **Memory Issues:** If encountering OOM (Out of Memory) errors, reduce the `batch_size` in the CFG class.
- **Plugin Warnings:** TensorFlow-IO may throw warnings regarding `.so` plugins; these are generally non-critical for standard file reading on Colab.

8. AI usage

During development, large language models were frequently used. Most common usages were model architecture refinement suggestion, code readability improvement through comments, and debugging support.